

PROJET NETWORK SNIFFING

Analyse Réseau avec Wireshark

Documentation Technique Complète

Découverte des protocoles réseau & analyse de paquets

Auteur : Berdejo Alexandre

École : La Plateforme

Date : Mai 2026

Version 1.0

Compétences : Administration & Sécurité Réseaux



Sommaire

Introduction et présentation du projet	3
Théorie : Notions fondamentales	4
• Wireshark, paquet, trame, PCAP	
• Modèle OSI (7 couches)	
• Modèle TCP/IP (4 couches)	
• Protocoles : ARP, TCP, UDP	
• En-têtes et Three-Way Handshake	
Partie 1 : Mise en place et captures sur VM Windows	12
• Installation de VMware et Windows 11	
• Captures TCP, ARP, UDP, SSDP	
Partie 2 : Réseau local Debian (Client / Serveur)	18
• Configuration de deux VM Debian	
• Captures FTP, HTTPS/TLS, DHCP, DNS, mDNS	
Partie 3 : Analyse en ligne de commande avec tshark	24
• Filtres ARP, DNS, TCP	
• Sauvegarde et exploitation des captures	
Partie 4 (BONUS) : Détection d'intrusion & sécurité avancée	28
• Détection d'attaques avec Wireshark	
• Scan de ports, ARP spoofing, exfiltration	
• Bonnes pratiques de sécurité	
Conclusion et bilan	35

Introduction et présentation du projet

Contexte

Ce projet « Network sniffing » réalisé dans le cadre de la formation à La Plateforme a pour objectif de découvrir et de maîtriser l'utilisation de Wireshark, l'outil de référence pour l'analyse de protocoles réseau. Au-delà de la simple capture de paquets, il s'agit de comprendre en profondeur le fonctionnement des protocoles qui font tourner Internet.

Objectifs

- Comprendre le fonctionnement des couches réseau (OSI et TCP/IP)
- Capturer et analyser des trames avec Wireshark sur différents environnements
- Mettre en place un réseau local entre deux machines virtuelles Debian
- Identifier les protocoles, leurs en-têtes, et les flux de données
- Découvrir les implications de sécurité (données en clair vs chiffrées)
- Apprendre l'utilisation de tshark en ligne de commande

Compétences visées

Le projet s'inscrit dans le cadre du référentiel de compétences en administration et sécurité des infrastructures réseaux :

- Administrer et sécuriser des infrastructures réseaux
- Mesurer le niveau de sécurité d'un système
- Détecter et traiter les incidents
- Définir une politique de sécurité

Environnement de travail

Élément	Détails
Hyperviseur	VMware Workstation Pro 17
VM Partie 1	Windows 11 (2 CPU, 4 Go RAM, 84 Go disque)
VM Partie 2	2 × Debian 13 (Server + Client)
Analyseur	Wireshark 4.6.5 (Windows) et 4.4.15 (Debian)
CLI	tshark (inclus avec Wireshark)

Notions fondamentales

Qu'est-ce que Wireshark ?

Wireshark est un analyseur de protocoles réseau open-source, considéré comme la référence dans le domaine. Il permet d'examiner en temps réel le trafic circulant sur une interface réseau, de le décoder protocole par protocole, et de filtrer les paquets pour en extraire les informations pertinentes.

Lorsqu'on lance une capture, Wireshark enregistre chaque trame qui passe sur l'interface choisie. On peut ensuite explorer ces données couche par couche, du signal physique jusqu'aux données applicatives.

Paquet, trame et format PCAP

Terme	Définition
Paquet	Segment d'un message au niveau de la couche 3 (Réseau). Contient les adresses IP source/destination et les données.
Trame	Paquet encapsulé au niveau de la couche 2 (Liaison). Ajoute les adresses MAC. Types : Ethernet, Wi-Fi, PPP, ATM.
PCAP	Packet Capture. Format de fichier classique pour enregistrer les captures réseau.
PCAPNG	PCAP Next Generation. Version plus extensible et flexible, supporte plusieurs interfaces et plus de métadonnées. C'est le format moderne recommandé.

Le modèle OSI (7 couches)

Le modèle OSI (Open Systems Interconnection) est un cadre conceptuel qui standardise les fonctions d'un système de communication en sept couches. Chaque couche a un rôle bien défini et communique avec la couche supérieure et inférieure.

N°	Couche	Rôle	Équipements / Protocoles
7	Application	Interface avec l'utilisateur, services réseau	HTTP, FTP, DNS, SSH, Pare-feu
6	Présentation	Chiffrement, compression, encodage	TLS, SSL, JPEG, GZIP
5	Session	Établissement et gestion des sessions	NetBIOS, SOCKS
4	Transport	Fiabilité ou rapidité de la transmission	TCP, UDP
3	Réseau	Routage entre réseaux différents	IP, ICMP, Routeurs
2	Liaison de données	Adressage local (MAC), détection d'erreurs	Ethernet, Switch, Carte réseau
1	Physique	Transmission des bits sur le médium	Câbles, Hubs, Répéteurs

 **Astuce mnémotechnique : « Pour Les Réseaux, Tout Se Passe Automatiquement » (Physique, Liaison, Réseau, Transport, Session, Présentation, Application).**

Le modèle TCP/IP (4 couches)

Le modèle TCP/IP, plus pragmatique, regroupe les couches OSI en 4 niveaux. C'est celui qui est réellement implémenté dans Internet.

N°	Couche TCP/IP	Correspondance OSI	Exemples
4	Application	Couches 5, 6, 7	HTTP, DNS, FTP
3	Transport	Couche 4	TCP, UDP
2	Internet	Couche 3	IP, ICMP
1	Accès réseau	Couches 1 et 2	Ethernet, Wi-Fi

Protocole ARP (Address Resolution Protocol)

ARP est le protocole qui permet de traduire une adresse IP en adresse MAC sur un réseau local. Quand une machine veut envoyer un paquet à une IP, elle doit d'abord connaître l'adresse MAC correspondante.

Fonctionnement :

- La machine consulte son cache ARP (table de correspondance IP ↔ MAC)
- Si l'IP n'est pas dans le cache, elle envoie une requête ARP en broadcast : « Qui a 192.168.1.20 ? »
- La machine concernée répond avec son adresse MAC
- Le cache ARP est mis à jour pour les prochaines communications

En-tête ARP : 28 octets contenant le type de réseau, le type de protocole, les longueurs d'adresse, l'opération (requête ou réponse), les adresses MAC et IP de l'émetteur et du destinataire.

Protocole UDP (User Datagram Protocol)

UDP est un protocole de transport sans connexion. Il envoie les données sans vérifier qu'elles soient bien arrivées. C'est rapide, mais sans garantie.

Cas d'usage : appels VoIP, streaming vidéo, jeux en ligne, DNS — tout ce où la rapidité prime sur la fiabilité.

En-tête UDP : seulement 8 octets (port source, port destination, longueur, checksum).

Protocole TCP (Transmission Control Protocol)

TCP est un protocole de transport orienté connexion. Il garantit que les données arrivent dans l'ordre et sans perte, grâce à un système de numérotation des paquets et d'accusés de réception.

En-tête TCP : 20 octets minimum (ports source/destination, numéro de séquence, numéro d'accusé, flags URG/ACK/PSH/RST/SYN/FIN, taille de fenêtre, checksum).

Three-Way Handshake (poignée de main à trois temps)

Avant tout échange de données, TCP établit une connexion en trois étapes :

Étape	Sens	Message
1	Client → Serveur	[SYN] Seq=0 — demande d'ouverture de connexion
2	Serveur → Client	[SYN, ACK] Seq=0, Ack=1 — acceptation
3	Client → Serveur	[ACK] Ack=1 — connexion établie ✓

TCP vs UDP : comparaison

Critère	TCP	UDP
Connexion	Orienté connexion	Sans connexion
Fiabilité	Garantie	Sans garantie
Ordre des paquets	Maintenu	Non garanti
Vitesse	Plus lent	Très rapide
Taille en-tête	20 octets	8 octets
Usages	HTTP, FTP, SSH, mail	DNS, VoIP, streaming

Partie 1 : Mise en place et captures sur VM Windows

La première partie consiste à mettre en place une machine virtuelle Windows 11 sur VMware Workstation Pro 17, puis à y installer Wireshark pour effectuer les premières captures réseau.

1.1 Installation de la VM Windows 11

La VM est configurée avec 2 processeurs, 4 Go de RAM, 84 Go de disque, et un adaptateur réseau en mode NAT. Cette configuration permet d'avoir Internet tout en isolant la VM du réseau physique.

WireShark

C'est quoi WireShark ?

WireShark est un outil permettant de comprendre et d'analyser en temps réel les protocoles d'un réseau.

Trame et paquet, la différence ?

Un paquet est un segment d'un message plus important afin de faciliter le transport et la communication dans un réseau (couche 3). La trame est le résultat du paquet une fois qu'il est encapsulé. Il existe plusieurs types de trames : ATM, PPP, Ethernet, Wifi... (couche 2)

Format

Un fichier PCAP (**P**acket **C**apture) est un format de fichier utilisé pour enregistrer des données de capture réseau.
Un fichier PCAPNG (**P**acket **C**apture **N**ext **G**eneration) est un format plus extensible et plus flexible que PCAP.

Modèle de règle d'uniformisation

Pour permettre la bonne communication des machines dans un réseau, il est nécessaire de mettre en place des règles d'uniformisation telles que : la nécessité d'une adresse IP sur les machines d'un réseau ou encore l'utilisation de RJ45 comme connecteur pour les réseaux cuivre. Cependant il en existe bien d'autres et le modèle OSI (**O**pen **S**ystems **I**nterconnection) en est d'ailleurs un recueil qui regroupe ces règles en 7 niveaux aussi appelés couches.

Figure 1 — Configuration de la VM Windows 11 dans VMware Workstation Pro

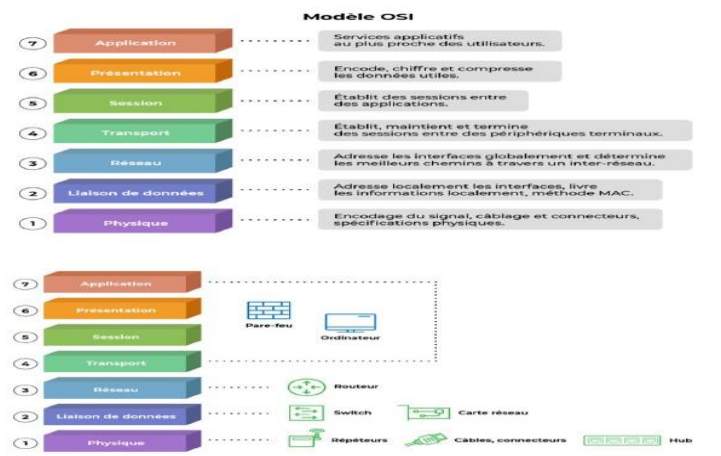
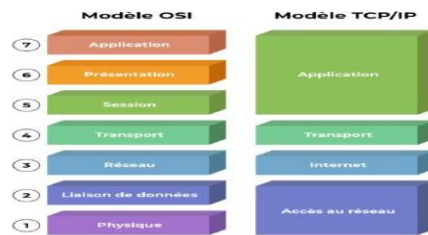


Figure 2 — Installation et démarrage de Windows 11

1.2 Installation de Wireshark sur Windows

Wireshark est téléchargé depuis le site officiel [wireshark.org](https://www.wireshark.org) (version 4.6.5, Windows x64 Installer).

Néanmoins il existe un modèle plus simple, plus applicable et comportant moins de couches, il s'agit du modèle TCP/IP



Ce modèle porte le nom des 2 protocoles les plus importants qui le constituent.

Le protocole IP (**I**nternet **P**rotocol) qui se situe dans la couche Réseau/Internet et le protocole TCP (**T**ransmission **C**ontrol **P**rotocol) qui lui se situe dans la couche Transport, son objectif étant d'établir des règles permettant de transporter un message de la source à la destination, en s'assurant que rien n'ait été perdu en route comme par exemple la règle de numérotation de chaque paquet d'un message. Lorsque un message est envoyé, il est découpé en paquet qui contiennent chacun, en plus du message, les règles d'uniformisation, c'est ce qu'on appelle l'encapsulation.



Les différents protocoles réseau

Figure 3 — Téléchargement de Wireshark 4.6.5 depuis le site officiel

Le protocole ARP (**A**ddress **R**esolution **P**rotocol) permet la communication de deux appareils en réseau local en traduisant leurs adresses IP en adresses MAC et inversement L'ARP est nécessaire parce que les adresses IP et MAC sont de longueurs différentes et fonctionnent sur des couches différentes du modèle OSI. L'ARP ne traduit les adresses que lorsqu'il s'agit de la première communication entre les deux appareils sinon il cherche dans le cache ARP du système hôte du réseau.

How Address Resolution Protocol (ARP) works



Le protocole UDP (**U**ser **D**atagram **P**rotocol) est un protocole fonctionnant avec l'IP et permettant de transmettre des données sensibles au temps telles que : des appels, des transmission en direct etc. Ce protocole envoie donc tous les paquets à la suite sans s'assurer qu'il ait été bien reçu

Et enfin, le protocole TCP (**T**ransmission **C**ontrol **P**rotocol) qui fonctionne lui aussi grâce au protocole IP permet lui cependant de transmettre des données de manière organisé en numérotant chaque paquets envoyés et en s'assurant de leurs arrivés

Figure 4 — Installation de Wireshark sur Windows 11

1.3 Première capture : analyse TCP

Une fois Wireshark lancé, on choisit l'interface réseau active et on applique le filtre tcp. On observe alors le trafic TCP en temps réel.

The differences between TCP and UDP	
Transmission Control Protocol	User Datagram Protocol
A connection-oriented protocol	A connectionless protocol
The most widely used protocol on the Internet	Used for voice over IP, streaming video, gaming and live broadcasts
Guarantees that no packets are missing and all the data that's sent makes it to the intended recipient	Faster and needs fewer resources
Sends packets in order so they can be stitched back together easily	Packets don't necessarily arrive in order
Slower and requires more resources	Allows missing packets, but the sender is unable to know whether a packet has been received
Has a bigger header than UDP, best suited for apps that need high reliability and transmission time is relatively less critical	Better suited for applications that need fast, efficient transmission, such as games

Figure 5 — Capture TCP : trafic TLSv1.2 entre 23.41.187.16 et 192.168.109.130

Détails observés dans le paquet IP :

- Version : 4 (IPv4)
- TTL : 128 hops max
- Protocole : TCP (numéro 6)
- IP source : 23.41.187.16
- IP destination : 192.168.109.130

Mise en place VM W

Figure 6 — Détail d'un paquet TCP : ports, séquence, flags

1.4 Capture du Three-Way Handshake

Le handshake TCP est visible en filtrant le trafic. On observe la séquence SYN → SYN-ACK → ACK qui initie chaque connexion TCP.

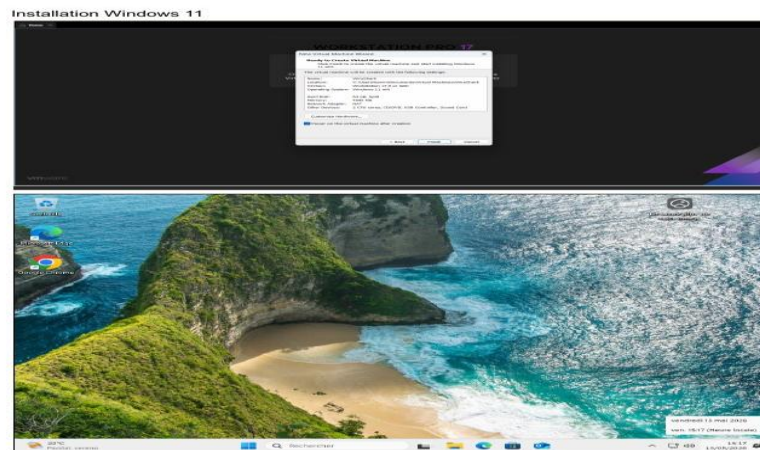


Figure 7 — Three-Way Handshake : 192.168.109.130 → 52.168.117.168 [SYN] Seq=0

1.5 Capture ARP

En filtrant sur arp, on observe les requêtes de résolution d'adresse. Le trafic ARP se fait en broadcast (adresse MAC ff:ff:ff:ff:ff:ff) pour interroger tout le réseau local.

Mise en place WireShark

Figure 8 — Requêtes ARP en broadcast depuis VMware_c0:00:08

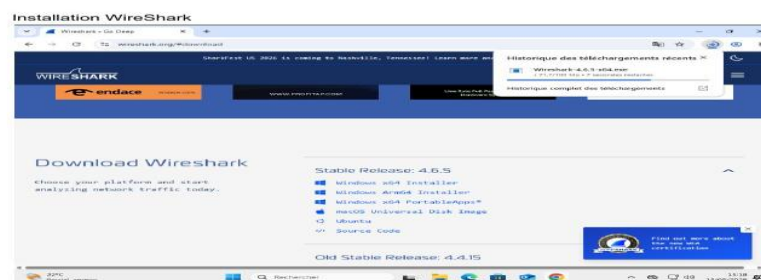


Figure 9 — Détail d'une requête ARP : « Who has 192.168.109.2? Tell 192.168.109.1 »

Informations extraites :

- Sender MAC : VMware_05:7c:c2 (00:0c:29:05:7c:c2)
- Sender IP : 192.168.109.130
- Target MAC : VMware_f0:52:77
- Opcode : request (1)

1.6 Capture UDP et SSDP

Le filtre `udp` permet de voir le trafic UDP, dont les annonces SSDP (Simple Service Discovery Protocol) émises en multicast vers 239.255.255.250 pour la découverte de services réseau (DLNA, UPnP, Chromecast).

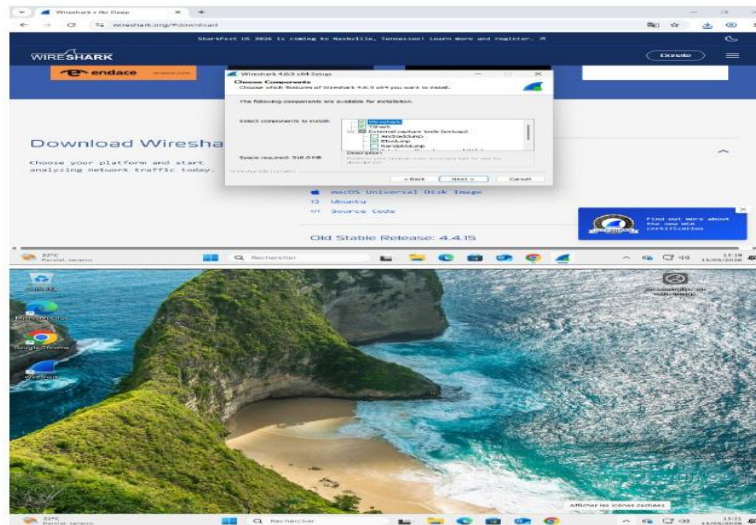


Figure 10 — Trafic SSDP vers 239.255.255.250 depuis 192.168.109.1

Partie 1

Figure 11 — M-SEARCH SSDP avec User-Agent Chromium/120.0

On observe également du trafic NBNS (NetBIOS Name Service) et DNS, ce qui montre la diversité des protocoles UDP utilisés en permanence par le système.

Partie 2 : Réseau local Debian (Client / Serveur)

La deuxième partie monte un véritable petit réseau local entre deux machines virtuelles Debian. L'une joue le rôle de serveur (WireSharkServer) avec différents services réseau installés, l'autre celui de client (WireSharkClient) qui sollicitera ces services. C'est cette interaction qui sera capturée et analysée.

2.1 Mise en place des deux VM Debian

Chaque VM est configurée identiquement : Debian 13 (debian-13.3.0-amd64-net), 2 Go de RAM, 2 processeurs, 20 Go de disque, et un adaptateur réseau NAT pour pouvoir communiquer entre les deux VM tout en gardant un accès Internet.

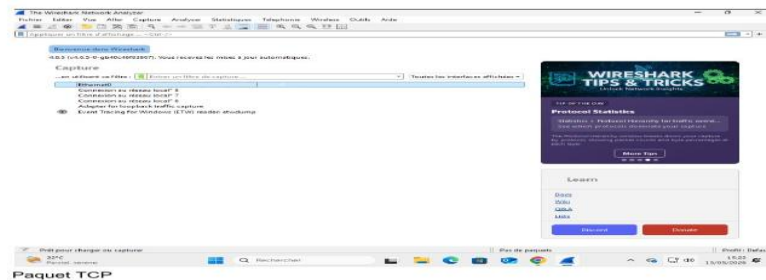


Figure 12 — Configuration de WireSharkServer dans VMware

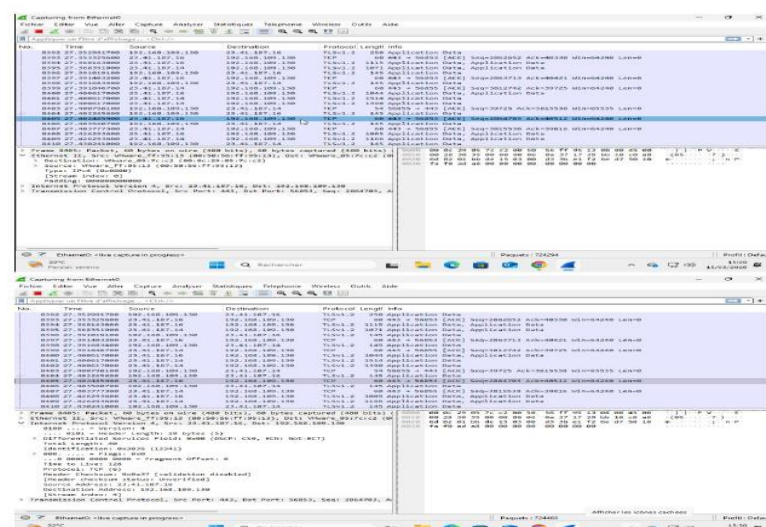


Figure 13 — Démarrage de Debian 13 — login root

Vérification de la communication entre les deux VM avec ping :

```
$ ping 192.168.109.133 # depuis le client vers le serveur 7 packets transmitted, 7 received, 0% packet loss
```

✓ 0 % de perte : les deux VM communiquent parfaitement. On peut commencer à mettre en place les services.

2.2 Installation des services sur le serveur

Sur WireSharkServer, on installe tous les services nécessaires pour générer du trafic réseau intéressant à analyser :

```
# apt install wireshark          # version 4.4.15 # apt install vsftpd          # serveur FTP #
apt install apache2             # serveur HTTP # apt install apache2-ssl        # support
HTTPS/TLS # apt install samba   # partage SMB # apt install dnsmasq      #
serveur DNS # apt install avahi-daemon # support mDNS
```

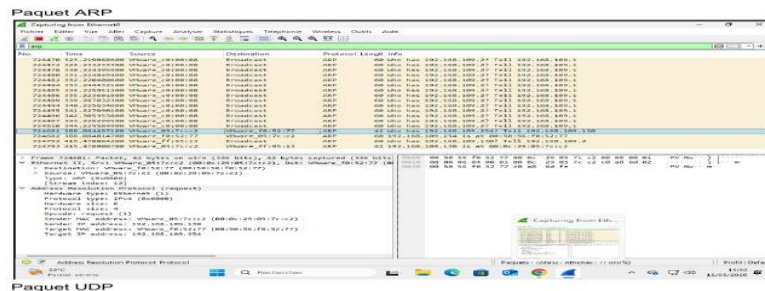


Figure 14 — Installation des paquets sur le serveur Debian

2.3 Capture FTP — données en clair ⚠

Le filtre ftp dans Wireshark révèle ce qui se passe lors d'une connexion FTP depuis le client. Et c'est inquiétant : tout passe en clair !

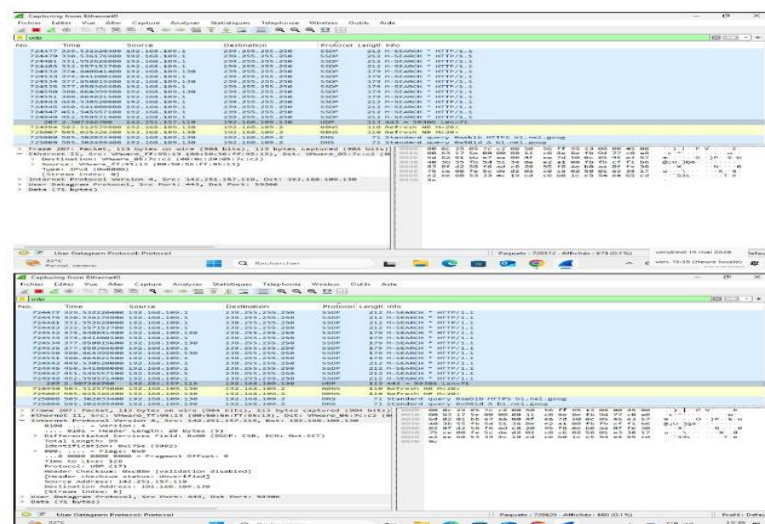


Figure 15 — Capture FTP : identifiants USER et PASS visibles en texte clair

Sur l'image, on voit clairement :

- Commande USER : test1
- Commande PASS : test
- Réponse serveur : « 230 Login successful »

 **CONSTAT DE SÉCURITÉ MAJEUR : Un attaquant qui sniffe le réseau peut récupérer les identifiants FTP en clair. Il ne faut JAMAIS utiliser FTP sur un réseau non sécurisé. Utiliser SFTP ou FTPS à la place.**

2.4 Capture HTTPS/TLS — données chiffrées

Maintenant, capturons le trafic vers la page Apache par défaut, mais cette fois en HTTPS. Filtre Wireshark : tls.

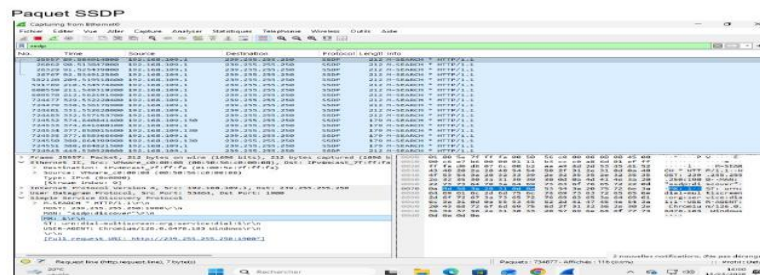


Figure 16 — Capture HTTPS/TLS : données entièrement chiffrées

Contrairement au FTP, ici Wireshark ne peut afficher que le fait qu'il y a une session TLSv1.3, mais le contenu lui-même est illisible. Pas d'identifiants, pas de contenu de page, juste du chiffré.

 **LEÇON CLÉ :** TLS protège efficacement la confidentialité des échanges. Toujours utiliser HTTPS pour les sites web, et préférer les versions chiffrées de tous les protocoles (SFTP, IMAPS, SMTPS, etc.).

2.5 Capture DHCP — la séquence DORA

Pour observer DHCP, on force le renouvellement d'IP sur le client avec `dhclient -r` puis `dhclient -v`. Filtre Wireshark : dhcp.

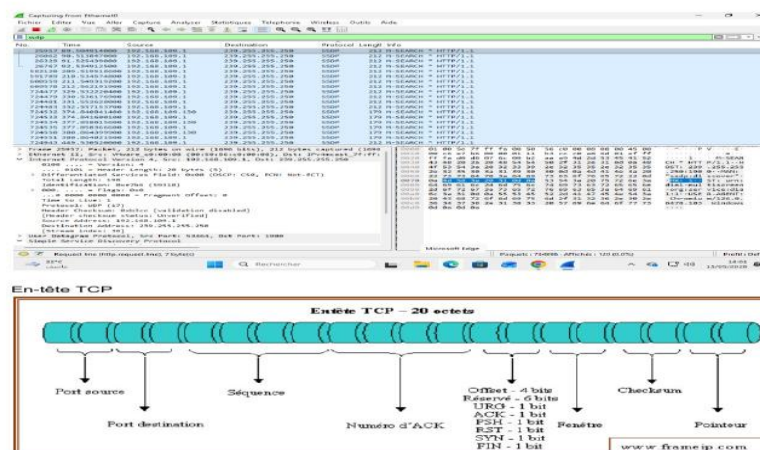


Figure 17 — Séquence DORA : Discover → Offer → Request → ACK

La séquence DORA est parfaitement visible :

- Discover — Le client cherche un serveur DHCP en broadcast
- Offer — Le serveur (192.168.109.254) propose une IP
- Request — Le client demande à confirmer cette IP
- ACK — Le serveur valide l'attribution

IP attribuée : 192.168.109.134, durée de bail (renewal) : 745 secondes.

2.6 Capture DNS

On lance nslookup google.com depuis le client pour générer une requête DNS. Filtre Wireshark : dns.

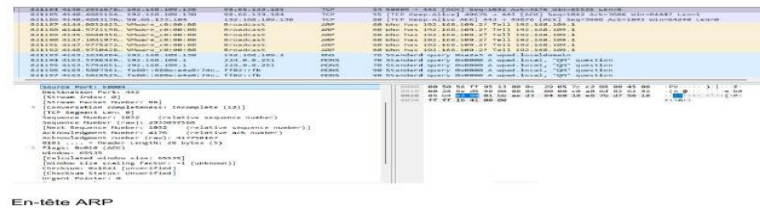


Figure 18 — Requête DNS pour google.com → 142.250.181.206

On voit :

- Requêtes de type A (IPv4) et AAAA (IPv6)
- Serveur DNS interrogé : 192.168.109.2
- Réponses pour google.com, fonts.gstatic.com, www.gstatic.com

2.7 Capture mDNS

Le filtre mdns permet de voir le trafic multicast DNS, utilisé notamment par Apple (Bonjour) et pour la découverte de Chromecast.

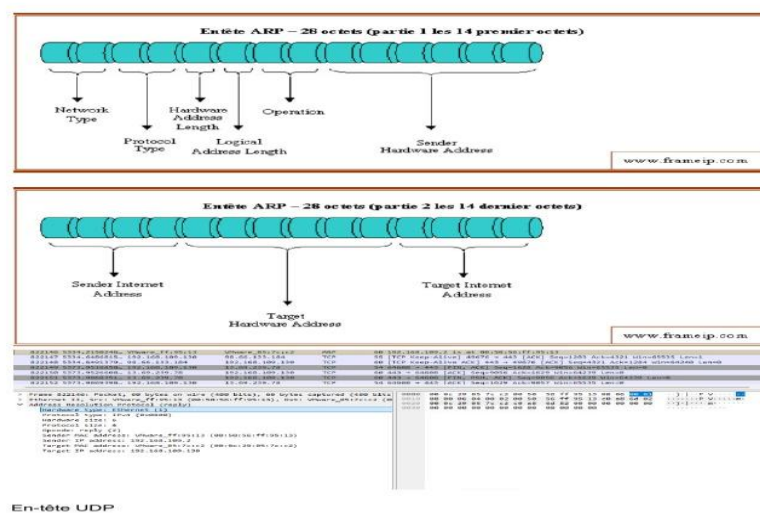


Figure 19 — Requêtes mDNS PTR vers 224.0.0.251 pour _googlecast._tcp.local

On distingue clairement les requêtes IPv4 et IPv6 (fe80::680e:e4a8:74c...) qui cherchent à découvrir les services disponibles sur le réseau local.

2.8 Bilan de la partie 2

Protocole	Sécurisé ?	Observations
FTP	✗ NON	Identifiants et données en clair
HTTPS/TLS	✓ OUI	Contenu entièrement chiffré
DHCP	⚠ Non	Broadcast, pas d'authentification (DHCP snooping nécessaire)
DNS	⚠ Non	Requêtes visibles (sauf DoH/DoT)
mDNS	⚠ Non	Multicast, expose les services locaux

Partie 3 : Analyse en ligne de commande avec tshark

La troisième partie passe en ligne de commande avec tshark, la version CLI de Wireshark. C'est l'outil indispensable pour automatiser les captures, les intégrer dans des scripts, ou travailler sur des serveurs sans interface graphique.

3.1 Capture ARP avec filtre

On lance une capture ARP en ligne de commande :

```
# tshark -i ens33 arp
```

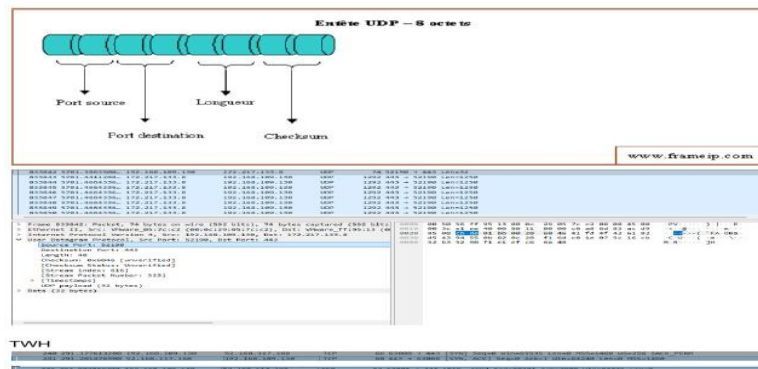


Figure 20 — Capture ARP : 4 paquets entre 192.168.100.10 et 192.168.100.20

On observe :

- Requête : « Who has 192.168.100.20? Tell 192.168.100.10 »
- Réponse avec MAC 00:0c:29:8c:ac:6e
- Puis la requête inverse depuis l'autre côté

3.2 Capture DNS avec filtre BPF

```
# tshark -i ens33 -f "udp port 53"
```

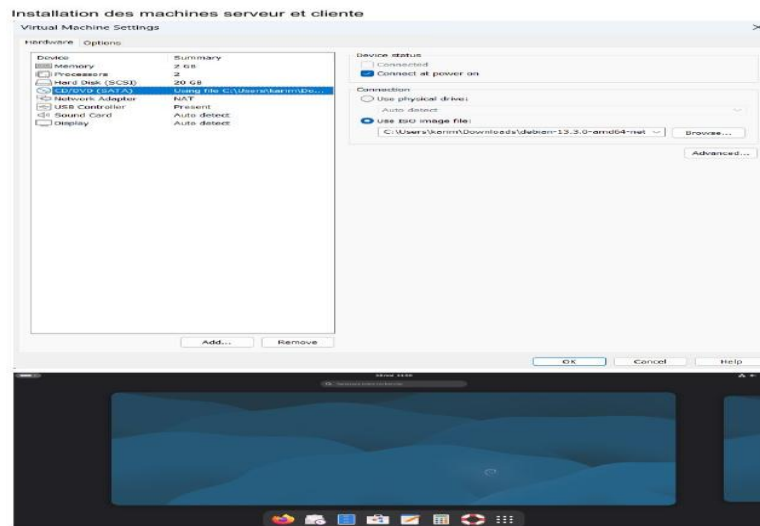


Figure 21 — Requêtes DNS : root servers et résolution google.com

On voit la requête vers les serveurs racine (a.root-servers.net, b.root-servers.net, etc.) puis la résolution finale de google.com → 142.250.181.206.

3.3 Capture du handshake SYN-ACK-FIN

```
# tshark -i ens33 -Y "tcp.flags.syn == 1 || tcp.flags.fin == 1"
```



Figure 22 — Capture filtrée sur les flags SYN et FIN

Détail du handshake observé :

- Paquet 1 : 192.168.100.1 → 192.168.100.20 port 22 [SYN] Seq=0 Win=65535 MSS=1460
- Paquet 2 : 192.168.100.20 → 192.168.100.1 [SYN, ACK] Seq=0 Ack=1 Win=64240 MSS=1460

3.4 Redirection des résultats vers des fichiers

tshark se prête parfaitement à la redirection pour conserver les captures pour analyse ultérieure :

```
# tshark -i ens33 arp > arp.txt # tshark -i ens33 -f "udp port 53" > dns.txt # tshark -i ens33 -f "tcp port 22" > ssh.txt
```

Partie 2

Figure 23 — Redirection des résultats vers des fichiers texte

3.5 Sauvegarde au format PCAPNG et relecture

Pour conserver les paquets en binaire et pouvoir les rouvrir dans Wireshark plus tard :

```
# tshark -i ens33 -w capture.pcapng # tshark -r capture.pcapng -Y arp # tshark -r capture.pcapng -Y "udp port 53"
```



Figure 24 — Sauvegarde et relecture filtrée d'un fichier PCAPNG



Figure 25 — Filtrage post-capture par protocole

💡 **ASTUCE PRO** : combiner `-w capture.pcapng` pour la sauvegarde et `-Y` filtres pour relire selon différents critères, c'est la méthode classique pour faire de l'analyse forensique.

3.6 Récapitulatif des commandes tshark utiles

Commande	Description
tshark -D	Lister les interfaces réseau disponibles
tshark -i ens33	Capture en direct sur l'interface ens33
tshark -i ens33 -c 10	Capturer seulement 10 paquets
tshark -i ens33 -f "tcp port 80"	Filtre de capture BPF
tshark -i ens33 -Y "http"	Filtre d'affichage Wireshark
tshark -i ens33 -w capture.pcapng	Sauvegarder la capture
tshark -r capture.pcapng	Relire une capture sauvegardée
tshark -r capture.pcapng -Y "arp"	Relire avec filtre
tshark -i ens33 -V	Affichage verbeux (détails complets)
tshark -i ens33 -x	Affichage hexadécimal des paquets

PARTIE BONUS

Partie 4 : Détection d'intrusion et sécurité avancée

 Cette partie va au-delà des consignes du projet. Elle propose d'utiliser Wireshark non plus comme un simple outil d'apprentissage des protocoles, mais comme un véritable outil d'analyse de sécurité. On y verra comment détecter des comportements suspects, identifier des attaques classiques, et adopter de bonnes pratiques.

4.1 Pourquoi cette partie ?

Wireshark n'est pas qu'un outil pédagogique. Dans le monde professionnel, il est utilisé quotidiennement par :

- Les analystes SOC (Security Operations Center) pour investiguer les incidents
- Les pentesters pour cartographier un réseau et détecter des vulnérabilités
- Les administrateurs réseau pour diagnostiquer des problèmes complexes
- Les développeurs pour déboguer des protocoles applicatifs

Cette partie présente trois cas d'usage concrets en sécurité offensive et défensive.

4.2 Détecter un scan de ports

Un scan de ports est souvent la première étape d'une attaque. Un attaquant cherche à identifier les services qui tournent sur une machine cible. Voici comment le détecter dans Wireshark.

Signature d'un scan TCP SYN (scan furtif Nmap)

Un scan SYN envoie un paquet [SYN] sur de nombreux ports et observe les réponses :

- Réponse [SYN, ACK] → le port est ouvert
- Réponse [RST, ACK] → le port est fermé
- Aucune réponse → le port est filtré (firewall)

Filtre Wireshark pour détecter ce type de scan :

```
tcp.flags.syn == 1 and tcp.flags.ack == 0
```

Indicateurs suspects :

- Énormément de SYN depuis une même IP source vers de nombreux ports différents
- Pas de Three-Way Handshake complet (jamais de ACK final côté attaquant)
- Beaucoup de paquets [RST] en réponse → ports fermés scannés en masse

Filtre pour observer les statistiques par IP

```
Wireshark → Statistics → Conversations → onglet IPv4
```

Si une IP a des centaines de petites conversations vers la même cible mais des ports différents, c'est une signature claire de scan.

4.3 Détecter une attaque ARP Spoofing

L'ARP spoofing (ou ARP poisoning) est une attaque de l'homme du milieu (MITM) très ancienne mais toujours efficace sur les réseaux locaux mal protégés. L'attaquant envoie de fausses réponses ARP pour faire croire à la cible qu'il est la passerelle.

Comment Wireshark révèle l'attaque

Filtre à utiliser :

```
arp.duplicate-address-detected or arp.duplicate-address-frame
```

Wireshark détecte automatiquement quand une même IP est associée à plusieurs adresses MAC différentes en peu de temps — c'est la signature exacte de l'ARP spoofing.

Indicateurs supplémentaires :

- Réponses ARP non sollicitées (gratuitous ARP) en grand nombre
- Une IP (typiquement celle de la passerelle) qui change soudainement de MAC
- Le cache ARP de la cible se met à jour vers la MAC de l'attaquant

⚠ CONTRE-MESURE : sur un switch managé, activer Dynamic ARP Inspection (DAI) couplé à DHCP snooping. Sur poste isolé, utiliser des entrées ARP statiques pour la passerelle critique.

4.4 Détecter une exfiltration de données via DNS

Le DNS tunneling est une technique d'exfiltration utilisée par des malwares avancés. L'idée : encoder les données volées dans les requêtes DNS, qui passent souvent sans inspection par les firewalls.

Signatures à observer

- Requêtes DNS de longueur anormalement importante (> 100 caractères)
- Sous-domaines aléatoires et nombreux : a3f7d9.exfil.attacker.com
- Type de requête TXT inhabituel pour de l'usage normal
- Volume de requêtes DNS bien au-dessus de la normale pour une machine donnée

Filtre Wireshark utile :

```
dns.qry.name.len > 50
```

Ou en tshark pour extraire les longueurs de requête :

```
tshark -r capture.pcapng -Y "dns" -T fields -e dns.qry.name -e dns.qry.name.len | sort -k2 -n
```

4.5 Identifier le trafic d'un malware (C2)

Les malwares communiquent souvent avec leur serveur de commande et contrôle (Command & Control, C2). Ce trafic a des signatures particulières observables dans Wireshark.

Indicateurs classiques

- Beaconsing : connexions régulières et espacées (toutes les X minutes) vers la même IP/domaine
- Ports non standards utilisés pour des protocoles HTTP/HTTPS (par exemple HTTPS sur 8443, 4443)
- Certificats TLS auto-signés ou suspects (analyser via le détail du Server Hello)
- Domaines générés par algorithme (DGA) : noms aléatoires comme xj4d8s2.com
- Trafic vers des ASN ou pays inhabituels pour l'organisation

Filtre Wireshark pour observer le beaconsing

Statistics → I/O Graphs permet de visualiser les pics de trafic et de repérer les patterns réguliers caractéristiques d'un beacon.

4.6 Bonnes pratiques de sécurité tirées du projet

Pour les utilisateurs et administrateurs

- Toujours préférer les protocoles chiffrés : HTTPS, SFTP, SSH, SMTPS, IMAPS
- Ne JAMAIS utiliser Telnet, FTP ou HTTP pour authentification ou données sensibles
- Activer HSTS sur les serveurs web pour forcer HTTPS
- Configurer DNS-over-HTTPS (DoH) ou DNS-over-TLS (DoT) pour chiffrer les requêtes DNS
- Désactiver les services non utilisés (SSDP, NetBIOS, mDNS) en environnement professionnel

Pour la sécurité du réseau local

- Segmenter le réseau avec des VLAN
- Activer DHCP snooping et DAI sur les switchs managés
- Mettre en place un IDS/IPS (Suricata, Snort, Zeek) qui consomme les captures
- Logger et conserver les captures pour analyse forensique post-incident
- Mettre à jour régulièrement les équipements réseau

Pour le monitoring continu

- Mettre en place un SIEM (Security Information and Event Management)
- Définir des baselines de trafic normal pour détecter les anomalies
- Automatiser les captures sur événement (déclencheur sur alerte)
- Former les équipes à l'analyse de paquets — c'est une compétence rare et précieuse

4.7 Outils complémentaires recommandés

Outil	Catégorie	Usage
tcpdump	Capture CLI	Alternative ultra-légère à tshark, présente sur tous les Unix
Nmap	Reconnaissance	Scan de ports et identification de services
Suricata	IDS/IPS	Détection d'intrusion temps réel basée sur règles
Zeek (Bro)	NSM	Network Security Monitoring, génère des logs détaillés
Scapy	Python	Crafting et manipulation de paquets en Python
NetworkMiner	Forensique	Extraction passive de fichiers et identifiants depuis un PCAP
MISP	Threat Intel	Plateforme de partage d'indicateurs de compromission

Conclusion et bilan

Ce qui a été appris

Ce projet « Network sniffing » a permis de couvrir un large spectre de compétences en analyse réseau :

- Compréhension théorique des modèles OSI et TCP/IP
- Maîtrise pratique de Wireshark et de tshark
- Capture et analyse de nombreux protocoles : ARP, TCP, UDP, DNS, DHCP, HTTPS, FTP, mDNS, SSDP
- Mise en place d'un réseau local fonctionnel entre deux VM Debian
- Identification claire des protocoles sécurisés vs non sécurisés
- Bases de l'analyse de sécurité offensive et défensive (partie bonus)

Le « moment WAOUH » du projet

La capture FTP a été le moment le plus marquant. Voir littéralement passer le mot de passe en clair sur le réseau a un impact pédagogique énorme. C'est une chose de lire « FTP n'est pas sécurisé » dans un livre, c'en est une autre de récupérer soi-même des identifiants à la volée en quelques secondes.

À l'inverse, la capture HTTPS a montré l'efficacité du chiffrement TLS : malgré le fait qu'on contrôle le réseau et qu'on capture tout le trafic, le contenu reste totalement illisible. La cryptographie marche.

Pistes pour aller plus loin

- Approfondir avec un IDS comme Suricata pour la détection automatique
- Apprendre Scapy pour créer des paquets à la main et tester des scénarios
- Faire des challenges CTF orientés network forensics (les PCAP challenges sont nombreux)
- Étudier des cas réels d'attaques via leurs captures publiées (Malware-Traffic-Analysis.net)
- Préparer des certifications réseau : CCNA, Network+, Security+, OSCP

Ressources et références

- Documentation officielle Wireshark : [wireshark.org](https://www.wireshark.org)
- RFC 826 (ARP), RFC 793 (TCP), RFC 768 (UDP), RFC 1035 (DNS)
- La Plateforme — Formation Administration et Sécurité Réseaux
- Site PracticalPacketAnalysis.com pour aller plus loin

Fin de la documentation — BERDEJO Alexandre — Mai 2026